

آزمایشگاه سیستم عامل

پروژه اول : آشنایی با هسته سیستم عامل لینوکس xv6

محمد صادق مهدوی پور 810102519

سید محمد جواد شریفی 810102469

محمد امین رشید 810102454

س ۱) وظیفه اول: به اشتراک گذاری منابع سخت افزار بین برنامه ها: سیستم عامل سخت افزار را بین چندین برنامه به صورت هم زمان به اشتراک می گذارد تا آن ها بتوانند به طور هم زمان اجرا شوند. برای مثال، پردازنده بین فرایندها تقسیم می شود تا هر کدام در زمان مناسب اجرا شوند.

وظیفه دوم: ساده سازی دسترسی به سخت افزار برای برنامه ها: سیستم عامل، سخت افزارهای سطح پایین مانند دیسک را مدیریت می کند تا برنامه ها بدون آنکه درگیر جزئیات فنی بشوند، بتوانند از آنها استفاده کنند. در واقع این اپلیکیشن ها نیازی به دانستن این جزئیات ندارند و فقط باید بتوانند از آنها استفاده کنند.

وظیفه سوم: ایجاد روش های کنترل شده برای ایجاد تعامل میان برنامه ها: سیستم عامل راه های امن و کنترل شده ای را برای اپلیکیشن ها ایجاد می کند تا بتوانند داده هایشان را با هم به اشتراک بگذارند. (مانند استفاده از file system)

س ۲) خیر – وجود سیستم عامل در همه دستگاه ها ضروری نیست. زیرا با توجه به وظایف سیستم عامل، می توان گفت که فقط زمانی حضور سیستم عامل لازم است که بخواهیم چند برنامه را به طور هم زمان اجرا کنیم، منابع را میان چند فرایند اشتراک گذاری و مدیریت کنیم و یا بخواهیم رابطی امن میان برنامه ها ایجاد کنیم. بنابراین در شرایطی که با دستگاه های ساده ای طرف هستیم یک برنامه ساده و تک منظوره دارند و نیازی به مدیریت و اشتراک گذاری ندارند، وجود سیستم عامل هم ضرورتی ندارد. در دستگاه هایی مانند کنترلرهای کوچک، وسایل خانگی هوشمند و مدارهایی با برنامه ثابت و تک منظوره، برنامه به طور مستقیم روی سخت افزار اجرا می شود و در این شرایط نیازی به حضور سیستم عامل نیست.

س ۳) در معماری xv6 می توان به نکات مهمی اشاره کرد که عبارتند از:

۱- استفاده از ساختار سنتی هسته: به این معناست که در xv6، هسته به عنوان بخش مرکزی سیستم است که تمام منابع سخت افزاری را مدیریت می کند و خدماتی را به برنامه ها ارائه می دهد.

۲- جدایی فضای کاربر و هسته: در xv6 دو سطح اجرایی وجود دارد: یکی فضای کاربر است که برنامه‌های معمول سیستم در این سطح اجرا می‌شوند. دومی فضای هسته است که در این سطح، کد خود هسته و با دسترسی بالا اجرا می‌شود. این ویژگی باعث حفاظت از منابع و امنیت می‌شود.

۳- تعامل از طریق system call ها: برنامه‌ها نمی‌توانند مستقیماً به سخت‌افزار دسترسی داشته باشند، بلکه از طریق فراخوانی‌های سیستمی (system calls) با هسته در ارتباط هستند.

در کل می‌توان بیان کرد که ساختار xv6 مانند Unix کلاسیک، یک سیستم‌عامل تک‌هسته‌ای است.

س ۴) سیستم‌عامل xv6 یک سیستم‌عامل چند وظیفه‌ای است زیرا می‌تواند زمان پردازنده را میان چند فرایند به طور اشتراکی تقسیم کند. در واقع یکی از وظایف هسته xv6، بحث scheduling است. Scheduling به معنای آن است که هسته، فرایندها را طوری زمان‌بندی می‌کند که هنگام اجرای یک فرایند، وضعیت فرایندهای دیگر ذخیره می‌شود و در زمان مناسب، اجرای آنها از سر گرفته می‌شود. همچنین وجود فرایندهای مانند فرایند والد و فرزند و سایر برنامه‌ها نشان‌دهنده چند وظیفه بودن xv6 است.

س ۵) تفاوت‌های برنامه و پردازنده به شرح زیر است:

۱- برنامه عبارت است از مجموعه‌ای از دستورالعمل‌ها که روی دیسک ذخیره شده‌اند. در حالیکه پردازنده، نمونه در حال اجرای یک برنامه در حافظه است.

۲- برنامه در حالت عادی، غیرفعال و ایستا است. (فقط کد است) اما پردازنده فعال است. (شامل کد، داده و پشته است).

۳- برنامه روی دیسک یا حافظه‌ای غیرفرار ذخیره می‌شود. در حالیکه پردازنده در RAM ذخیره و اجرا می‌شود.

۴- یک برنامه می‌تواند چندین بار اجرا شود که به هر اجرا از برنامه، یک پردازنده می‌گویند.

۵- یک برنامه تا زمان اجرا، نیازی به مدیریت OS ندارد اما پردازنده توسط OS مدیریت و زمان‌بندی می‌شود.

مثلاً در xv6 زمانی که دستور fork() اجرا می‌شود، سیستم‌عامل یک پردازنده جدید می‌سازد که یک کپی از حافظه برنامه والد است. در این شرایط، برنامه ثابت است اما یک پردازنده جدید با حافظه و شناسه مخصوص، تولید می‌شود.

س ۶) هر پردازنده دارای ۳ بخش اصلی است:

۱- instructions: دستورالعمل‌هایی که پردازنده اجرا می‌کند.

۲- data: متغیرها و داده‌های برنامه.

۳- stack: ذخیره متغیرهای local و مقدار بازگشت توابع.

همچنین برای هر پردازش در هسته اطلاعاتی نگهداری می‌شود. از جمله رجیسترهای CPU، شناسه پردازش، وضعیت اجرا و جدول file descriptors.

نحوه اختصاص CPU به پردازش‌ها به صورت time sharing و با استفاده از scheduling و context switching است. Scheduling به معنای این است که هسته xv6 به صورت چرخشی، بین پردازش‌های آماده اجرا سوئیچ می‌کند و وقتی یک پردازش در حال اجرا نیست، xv6 وضعیت رجیسترهای آن را ذخیره کرده و پردازش بعدی را اجرا می‌کند.

همچنین context switch به معنای آن است که هنگام جابجایی بین پردازش‌ها، مقادیر رجیسترهای CPU و شمارنده‌ی برنامه‌ی هر پردازش ذخیره و هنگام بازگشت دوباره بازیابی می‌شوند تا پردازش از همان نقطه ادامه دهد.

س ۷) در سیستم‌های مبتنی بر UNIX، file descriptor یک عدد صحیح کوچک است که برای ارجاع به یک منبع I/O، به هر پردازش می‌دهد. هر پردازش در xv6 یک جدول file descriptor مخصوص دارد که از صفر شروع می‌شود. مقدار ۰، به معنای ورودی استاندارد (stdin)، مقدار ۱، به معنای خروجی استاندارد (stdout) و مقدار ۲، به معنای خطای استاندارد (stderr) است.

Pipe یک بافر کوچک در هسته است که برای ارتباط بین پردازش‌ها استفاده می‌شود. در xv6 دستور pipe(p) یک کانال ارتباطی دو طرفه میان دو پردازش ایجاد می‌کند که در p[1] شناسه سمت نوشتن و در p[0] شناسه سمت خواندن ذخیره می‌شود. هر چیزی که در سمت نوشتن نوشته شود، در سمت خواندن قابل خواندن است. (این شناسه‌ها همان شماره‌های file descriptor ها هستند.) بنابراین هدف اصلی استفاده از pipe برای ارتباط و تبادل داده بین دو یا چند پردازش و ایجاد یک pipeline در اجرای هم زمان دستورات است.

س ۸) در سیستم عامل xv6، فراخوانی سیستمی fork()، برای ایجاد یک پردازش جدید (فرزند) به کار می‌رود که کپی دقیقی از پردازش والد است و دارای PID مجزا می‌باشد اما فراخوانی exec()، حافظه‌ی پردازش فعلی را با محتوای یک برنامه‌ی جدید جایگزین کرده و اجرای آن را از ابتدا آغاز می‌کند، بدون آنکه شناسه‌ی پردازش تغییر کند.

جدا بودن این دو فراخوانی یکی از اصول طراحی سیستم‌های UNIX است. زیرا این جداسازی اجازه می‌دهد پیش از اجرای برنامه‌ی جدید، محیط پردازش فرزند تغییر یابد؛ مثلاً ورودی و خروجی‌ها تنظیم یا فایل‌های خاصی باز و بسته شوند. به همین دلیل، تفکیک fork() و exec() موجب انعطاف‌پذیری، سادگی و قدرت بیشتر در اجرای هم‌زمان برنامه‌ها (مانند پیاده‌سازی shell) در xv6 شده است.

```
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-pie -fno-pic -nostdinc -I. -c entry.o
ld -m elf_i386 -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
objcopy -S -O binary -j .text bootblockother.o entryother
objdump -S bootblockother.o > entryother.asm
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-pie -nostdinc -I. -c initcode.S
ld -m elf_i386 -N -e start -Ttext 0 -o initcode.out initcode.o
objcopy -S -O binary initcode.out initcode
objdump -S initcode.o > initcode.asm
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o file.o fs.o ide.o ioapic.o kalloc.o kbd.o lapic.o log.o main.o mp.o picirq.o pipe.o proc.o sleeplock.o spinlock.o string.o switch.o syscall.o sysfile.o sysproc.o trapasm.o uart.o vectors.o vm.o -b binary initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel.sym
dd if=/dev/zero of=xv6.img count=10000
dd if=bootblock of=xv6.img conv=notrunc
dd if=kernel of=xv6.img seek=1 conv=notrunc
mohammad@mohammad-VirtualBox:~/OS/OS-Lab-Fall-2025/xv6$
```

دستور فوق، هسته نهایی سیستم عامل را می سازد.

با زدن دستور make، مجموعه ای از فایل های منبع کامپایل می شوند و به فایل های شیء (object files) با پسوند O. تبدیل می گردند. اما این فایل ها به تنهایی قابل اجرا نیستند و باید با هم ترکیب شوند تا یک فایل باینری واحد ساخته شود. دستور ld فایل های O. را با هم ترکیب کرده و یک فایل باینری قابل اجرا می سازد. بنابراین در طی این دستور، همه ی اجزای هسته در کنار هم قرار می گیرند و سیستم عامل به شکل یک برنامه ی قابل اجرا روی سخت افزار درمی آید. پس می توان گفت که این دستور، هسته نهایی سیستم عامل را می سازد.

اضافه کردن یک متن به Boot Message :

در این بخش اسامی خودمان را در فایل init.c پرینت میکنیم. این تابع وقتی اجرا میشود که کنسول qemu باز میشود.

```
14
15
16 if(open("console", O_RDWR) < 0){
17     mknod("console", 1, 1);
18     open("console", O_RDWR);
19 }
20 dup(0); // stdout
21 dup(0); // stderr
22
23 for(;;){
24
25
26
27     printf(1, "Sadegh Mahdavi\n");
28     printf(1, "javad sharifi\n");
29     printf(1, "Mohammad Amin Rashid\n");
30
31     printf(1, "init: starting sh\n");
32     pid = fork();
33     if(pid < 0){
34         printf(1, "init: fork failed\n");
35         exit();
36     }
37     if(pid == 0){
38         exec("sh", argv);
39         printf(1, "init: exec sh failed\n");
40         exit();
41     }
42     while((wpid=wait()) >= 0 && wpid != pid)
43         printf(1, "zombie!\n");
44 }
45 }
```

```

Machine  View
SeaBIOS (version 1.16.3-debian-1.16.3-2)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFCB050+1EF0B050 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
Sadeqh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$

```

اضافه کردن قابلیت های جدید به کنسول:

1. ابتدا کد هگزادسیمال کلید چپ و راست را تعریف میکنیم.

```

#define KEY_LEFT    0xE4
#define KEY_RIGHT   0xE5

```

سپس در در تابع `consoleintr` که کاراکتر مورد نظر را از `kbd.c` میگیرد ، این 2 مورد را به صورت زیر اضافه میکنیم:

`input.e` ایندکسی است که نشان میدهد که کرسر روی کدام یک از کاراکتر های بافر است.

```
switch(c){
    case KEY_LEFT:
        if(input.e > input.w){
            consputc(KEY_LEFT);
            input.e--;
        }

        break;

    case KEY_RIGHT:
        if(input.e < input.end_pos){
            consputc(KEY_RIGHT);
            input.e++;
        }

        break;
```

سپس در تابع `cgaputc` که مسئولیت نمایش در رابط ترمینال `qemu` را دارد ، این 2 مورد را اضافه میکنیم تا کرسر موجود در ترمینال جابه جا شود که این از تغییر `pos` استفاده میشود.

```
else if (c == KEY_RIGHT)
{
    pos++;
    outb(CRTPORT + 1, pos);
    return;
}
else if (c == KEY_LEFT)
{
    --pos;
    outb(CRTPORT + 1, pos);

    return;
}
```

هم چنین `input.end_pos` را به `struct` اضافه میکنیم که نشان دهنده ماکسیمم ایندکسی است که `input.e` تا آنجا رفته است ، از این در جاهای متعددی استفاده میکنیم مثلا اگر به چپ میرویم باید بدانیم چقدر میتونیم به راست بیاییم.همینطور کاربرد های متعدد دیگری دارد و آخر دستور را همیشه نگه میدارد.

همچنین این متغیر را در جا های مناسب آپدیت میکنیم، مثلا وقتی بک اسپیس میزنیم ، این متغیر یکی کم میشود.

با انجام این کار 2 مشکل اساسی پیش می آید. یکی این است که هنگامی که در وسط خط باشیم یعنی با KEY LEFT به چپ بیاییم :

اگر Backspace بزنیم، باید کاراکترهای سمت راست کرسر یکی به چپ شیفت داده بشوند:

```
break;
case C('H'): case '\x7f': // Backspace
    if(input.e != input.w){
        if(input.e < input.end_pos ){

            input.e--;

            consputc(BACKSPACE);

            move_chars_left();
            input.end_pos--;

        }
        else{
            input.e--;
            input.end_pos--;

            consputc(BACKSPACE);
        }
    }
    break;
```



```
void move_chars_left(){
    for (uint i=input.e ; i<input.end_pos-1 ; i++){

        input.buf[i % INPUT_BUF]=input.buf[(i+1)%INPUT_BUF];
        consputc(input.buf[i % INPUT_BUF]);

    }

    // uartputc(' ');
    for (uint i=input.e ; i<input.end_pos-1 ; i++){
        move_cursor(-1);
        // uartputc('\b');
    }

    return;
}
```

اگر کاراکتری تایپ کردیم باید کاراکترهای سمت راست کرسر یک واحد به راست شیفت داده

```
default:
if(c != 0 && input.e-input.r < INPUT_BUF){
    c = (c == '\r') ? '\n' : c;

    if((input.e < input.end_pos) && c!='\n'){
        for(int i=0 ; i<INPUT_BUF ; i++){
            copy_buf[i]=input.buf[i];
        }
        input.buf[input.e++ % INPUT_BUF] = c;
        consputc(c);

        move_chars_right();

        input.end_pos++;
        break;
    }
    input.buf[input.e++ % INPUT_BUF] = c;
    if(c == '\n' || c == C('D') || input.e == input.r+INPUT_BUF){ //line complete
        input.w = input.e;
        x=0;
        input.end_pos=input.e;
        wakeup(&input.r);
        break;
    }
    if(input.e==input.end_pos+1){
        input.end_pos++;
        consputc(c);
    }
}
```

```
void move_chars_right(){

    for (uint i=input.e ; i<=input.end_pos ; i++){

        input.buf[(i) % INPUT_BUF]=copy_buf[(i-1)%INPUT_BUF];
        consputc(input.buf[(i) % INPUT_BUF]);

    }

    for (uint i=input.e ; i<=input.end_pos; i++){

        move_cursor(-1);
        uartputc('\b');

    }
}
```

بشوند:

```

case C('D'):

    int flag=0;
    int found=0;
    int step=0;
    int j=input.e;
    while(j<= input.end_pos ){
        if(flag==1 && input.buf[j % INPUT_BUF]!=' '){
            found=1;
            break;
        }

        if(input.buf[j % INPUT_BUF]==' '){
            flag=1;
        }
        j++;
        step++;
    }
    if(found){
        for(int i=0 ; i<step ; i++){
            move_cursor(1);
            input.e++;
        }
    }

    break;
case C('^'):

```

بخش بعدی ctrl D است که باید به اول کلمه بعد برود:

از input.e شروع میکنیم و تا زمانی که به input.endpos نرسیدیم ، اولین ایندکس کاراکتری که بعد از اسپیس شروع میشود را پیدا میکنیم و کرسر را آنجا قرار میدهیم و e را آپدیت میکنیم.

بخش بعد ctrl A :

در اینجا دو حالت داریم که با توجه به اینکه در کدام حالت هستیم ، تابع مناسب فراخوانی میشود و مشابه بخش قبل کرسر و e را در جای درست میبریم:

```
case C('A'):  
  
    if(input.e != input.w){  
  
        int j=input.e;  
        if(input.buf[j % INPUT_BUF]==' ' || input.buf[(j-1) % INPUT_BUF]==' '){  
            move_to_first_previous();  
        }  
        else{  
            move_to_first_current();  
        }  
    }  
    break;
```

```
void move_to_first_previous(){  
    int flag=0;  
    int found=0;  
    int step=0;  
    int j=input.e;  
  
    while(j > input.w ){  
        if(flag==2 && input.buf[j % INPUT_BUF]==' '){  
            move_cursor(1);  
            input.e++;  
            found=1;  
            break;  
        }  
        if(flag==1 && input.buf[j % INPUT_BUF]!=' '){  
            flag=2;  
        }  
  
        if(input.buf[j % INPUT_BUF]==' ' && flag==0){  
            flag=1;  
        }  
        move_cursor(-1);  
        input.e--;  
        j--;  
        step++;  
    }  
    return;  
}
```

```

6 void move_to_first_previous(){
7     int flag=0;
8     int found=0;
9     int step=0;
10    int j=input.e;
11
12    while(j > input.w ){
13        if(flag==2 && input.buf[j % INPUT_BUF]==' '){
14            move_cursor(1);
15            input.e++;
16
17            found=1;
18            break;
19        }
20        if(flag==1 && input.buf[j % INPUT_BUF]!=' '){
21            flag=2;
22        }
23
24        if(input.buf[j % INPUT_BUF]==' ' && flag==0){
25            flag=1;
26        }
27        move_cursor(-1);
28        input.e--;
29        j--;
30        step++;
31    }
32
33    return;
34 }

```

۲) اضافه کردن قابلیت پاک کردن آخرین کاراکتر با `ctrl + z` :

برای این کار یک آرایه جدید به طول آرایه اصلی ساخته ایم که ساختار هایی که در آن خود کاراکتر و زمان ورودی آن وجود دارد در آن هستند. در ساختار اصلی ورودی هم یک زمان اضافه کرده ایم تا زمان آخرین کاراکتر وارد شده را همیشه داشته باشیم. در حین ای که کاربر ورودی میدهد یک نمونه از ساختار `char_time` ساخته میشود که زمان جدید کاراکتر وارد شده را در آن به همراه

خود کاراکتر ذخیره میکند.

```
input.buf[input.e++ % INPUT_BUF] = c;
if (input.e == input.end_pos + 1)
{
    input.end_pos++;
    consputc(c);

    uint last_char_time_position_index = input.e - input.w - 1;
    struct char_time new_char;
    new_char.c = c;
    new_char.time = input.time++;
    times_buf[last_char_time_position_index] = new_char;
}
}
break;
```

حالا index مربوط به آخرین حرف ذخیره شده در آرایه `timed_chars` را پیدا میکنیم و جای درست را برای ساختار ایجاد شده جدید پیدا میکنیم و ذخیره میکنیم (چون در این حالت کاربر کاراکتر وارده را در انتهای رشته وارد میکند و `cursor` در سمت راست رشته ورودی است).

در حالت زیر چون کاربر در وسط رشته ورودی میدهد باید `index` درست در بافر اصلی و بافر ساخته شده برای آپدیت بافر پیدا شود که کاراکتر جدید به آن افزوده شود. برای این کار از فرمول های داخل عکس استفاده میشود. قبل از افزودن کاراکتر به بافر ها باید از آن `index` به راست همه یک شیفت به راست داشته باشند. سپس کاراکتر جدید در بافر قرار میگیرد.

در نهایت زمان کل یکی اضافه میشود تا کاراکتر بعدی زمان یکسان با کاراکتر فعلی نداشته باشد.

```

if ((input.e < input.end_pos) && c != '\n')
{
    for (int i = 0; i < INPUT_BUF; i++)
    {
        copy_buf[i] = input.buf[i];
    }
    input.buf[input.e++ % INPUT_BUF] = c;
    consputc(c);
    move_chars_right();

    // right shift needs for times_buf befor inserting new object
    uint new_char_time_position_index = input.e - input.w - 1;
    uint last_char_time_position_index = (input.end_pos - 1) - input.w;
    move_timed_chars_right(new_char_time_position_index);
    struct char_time new_char;
    new_char.c = c;
    new_char.time = input.time++;
    times_buf[new_char_time_position_index++] = new_char;

    input.end_pos++;
    break;
}

```

در اینجا بعد از فشردن enter باید بافر timed_chars پاک شود برای دستور بعدی.

```

if (c == '\n' || c == C('D') || input.e == input.r + INPUT_BUF)
{
    input.buf[input.end_pos++ % INPUT_BUF] = c;
    input.w = input.end_pos;
    input.e = input.end_pos;
    wakeup(&input.r);

    if (c == '\n')
    {
        clear_char_time_array();
        input.time = 0;
    }

    break;
}

```

پس از فشردن `ctrl + z` این کیس اجرا خواهد شد. در ابتدا شرط `select` چک میشود که اگر قسمتی از ورودی انتخاب شده باشد با فشردن `ctrl + z` آن قسمت `deselect` میشود. سپس مکان کاراکتری که باید حذف شود ذخیره میشود. تابعی که این را به ما میدهد با پیدا کردن `index` کاراکتری از بافر که بیشترین زمان را دارد کار میکند. سپس مکان `cursor` و `index` واقعی کاراکتر در بافر اصلی را نیز بدست می آوریم. سپس چک میشود که اگر ورودی خالی بود مجدداً بافر زمان دار ما خالی شود و زمان کل هم صفر شود.

```
case C('Z'):  
  
    if (input.mode == 2)  
    {  
        print_select(input.s1, input.s2);  
        input.mode = 0;  
        break;  
    }  
  
    uint removing_char_index = find_max_char_time_index();  
    uint interval = input.end_pos - input.w;  
    uint cursor_index = input.e - input.w - 1;  
    uint absolute_char_index = input.w + removing_char_index;  
  
    if (input.end_pos == input.w)  
    {  
        clear_char_time_array();  
        input.time = 0;  
        break;  
    }  
}
```

در زیر برای پیدا کردن و پاک کردن کاراکتر باید نشانگر را به مکان آن ببریم که با توجه به اینکه نشانگر قبل مکان آن یا بعد از آن باشد این مساله تفاوت میکند. ابتدا نشانگر را به بعد از مکان کاراکتر می ببریم و سپس بعد از پاک کردن آن به جای اصلی آن باز میگردانیم.


```

else if (cursor_index > removing_char_index)
{
    for (uint i = cursor_index; i > removing_char_index; i--)
        move_cursor(-1);

    consputc(BACKSPACE);
    move_timed_chars_left(removing_char_index, absolute_char_index);

    for (uint i = cursor_index; i > removing_char_index; i--)
        move_cursor(1);

    input.e--;
}
else if (cursor_index < removing_char_index && removing_char_index < interval)
{
    for (uint i = cursor_index; i < removing_char_index; i++)
        move_cursor(1);

    consputc(BACKSPACE);
    move_timed_chars_left(removing_char_index, absolute_char_index);

    for (uint i = cursor_index; i < removing_char_index - 1; i++)
        move_cursor(-1);
}

```

اگر نشانگر هم روی مکان کاراکتر بود فقط لازم است آن را پاک کنیم و همه کاراکترهای بعد از آن را یکی به چپ شیفت بدهیم.

در نهایت هم `end_pos` یکی کم خواهد شد زیرا یکی از کاراکترهای ما حذف شده است.

```

else
{
    consputc(BACKSPACE);
    move_timed_chars_left(removing_char_index, absolute_char_index);

    input.e--;
}
input.end_pos--;
break;

```

3) اضافه کردن قابلیت سوم برای select و copy paste کردن و سایر کارها:

```
case C('S'):  
    if (input.mode == 0)  
    {  
        input.s1 = input.e;  
        input.mode = 1;  
    }  
    else if (input.mode == 1)  
    {  
        input.mode = 2;  
        input.s2 = input.e;  
        input.color = 'W';  
        print_select(input.s1, input.s2);  
        input.color = 'B';  
    }  
    else if (input.mode == 2)  
    {  
        print_select(input.s1, input.s2);  
        input.mode = 0;  
    }  
  
    break;
```

Mode همانطور که معلوم است نشان میدهد که وضعیت سلکت به چه صورت است. اگر دوبار سلکت کنیم یعنی رشته ای را سلکت کنیم، این مود به 2 تغییر میکند.

در این تابع، تابع print_select فراخوانی میشود. وظیفه این تابع این است که بخش select شده را پرینت کند. در این حالت ما color را سفید میکنیم و پس از فراخوانی بخش سلکت شده سفید میشود. از این تابع برای دی سلکت کردن هم استفاده میکنیم زیرا میخواهیم رنگش به حالت اولیه برگردد.

```

void print_select(uint s1, uint s2)
{
    uint min, max;
    if (s1 < s2)
    {
        min = s1;
        max = s2;
    }
    else
    {
        min = s2;
        max = s1;
    }
    int delta = (int)(min - input.e);
    move_cursor(delta);

    for (uint i = min; i <= max; i++)
    {
        consputc(input.buf[i % INPUT_BUF]);
    }

    if (s1 > s2)
    {
        for (uint i = min; i <= max; i++)
        {
            move_cursor(-1);
        }
    }
    else
    {
        move_cursor(-1);
    }
}

```

تابع زیر بخش select شده را کامل حذف میکند از این تابع در بخش back space در حالت سلکت شده و تایپ یک کاراکتر دیگر استفاده میکنیم:

```

void delete_selected()
{
    input.mode = 0;
    if (input.s1 <= input.s2)
    {
        move_cursor(1);
        input.e++;
    }
    else
    {
        uint temp;
        temp = input.s1;
        input.s1 = input.s2;
        input.s2 = temp;
        move_cursor(1 + (int)(input.s2 - input.s1));
        for (int i = 0; i <= (int)(input.s2 - input.s1); i++)
        {
            input.e++;
        }
    }
    for (int i = 0; i <= (int)(input.s2 - input.s1); i++)
    {
        if (input.e < input.end_pos)
        {
            input.e--;
            consputc(BACKSPACE);
            move_chars_left();
            input.end_pos--;
        }
        else
        {
            input.e--;
            input.end_pos--;
            consputc(BACKSPACE);
        }
    }
    return;
}

```

برای ctrl C هم از یک آرایه کمکی استفاده میکنیم که پس از کپی کردن کل بافر را در آن کپی میکنیم و دو تا اندیس مشخص میکنیم که نشان دهنده همان رشته انتخاب شده است.

و is_copy را یک میکنیم:

```
case C('C'):
    if (input.mode == 2)
    {
        is_copy = 1;
        for (int i = 0; i < INPUT_BUF; i++)
        {
            ctrl_c[i] = input.buf[i];
        }
        if (input.s1 <= input.s2)
        {
            inedx_copy1 = input.s1 % INPUT_BUF;
            inedx_copy2 = input.s2 % INPUT_BUF;
        }
        else
        {
            inedx_copy1 = input.s2 % INPUT_BUF;
            inedx_copy2 = input.s1 % INPUT_BUF;
        }
    }

    break;
```

برای کنترل V کافی است تمامی کاراکترهای ذخیره شده در بخش کنترل C را در ترمینال بنویسیم. توجه داریم که اگر رشته ای در این حین سلکت شده باشد ، یکبار با فراخوانی delete selected کل کاراکتر هارا پاک میکنیم ، سپس ادامه میدهیم.

```

3         break;
4     case C('V'):
5
6         if (is_copy == 1)
7         {
8             if (input.mode == 2)
9             {
10                 delete_selected();
11             }
12
13             for (uint i = inedx_copy1; i <= inedx_copy2; i++)
14             {
15                 c = ctrl_c[i];
16                 if ((input.e < input.end_pos) && c != '\n')
17                 {
18                     for (int i = 0; i < INPUT_BUF; i++)
19                     {
20                         copy_buf[i] = input.buf[i];
21                     }
22                     input.buf[input.e++ % INPUT_BUF] = c;
23                     consputc(c);
24
25                     move_chars_right();
26
27                     input.end_pos++;
28                     continue;
29                 }
30
31                 input.buf[input.e++ % INPUT_BUF] = c;
32                 if (input.e == input.end_pos + 1)
33                 {
34                     input.end_pos++;
35                     consputc(c);
36                 }
37             }
38         }
39     },

```

برای سایر دستورات به صورت زیر رشته deselect میشود:

```

case C('Z'):
    if (input.mode == 2)
    {
        print_select(input.s1, input.s2);
        input.mode = 0;
        break;
    }

```

4) دو تا دستور جدید:

دستور اول) با زدن فلش های بالا و پایین ، دستورات قبلی و بعدی که ما در ترمینال زده ایم ،نوشته شود.پیاده سازی به این صورت است که هر بار کاربر Enter میزند، آن خط کامل در یک جایی ذخیره میشود (تاریخچه محاسباتی) و با زدن فلش ها بتواند بین دستور ها عقب و جلو برود.

دستور دوم) ابتدا کاربر دستور ctrl F را میرند ولی اتفاقی در کنسول نمی افتد.سپس میتواند کاراکتری را در کیبورد بزند.با این کار cursor باید از اول خط روی اولین کاراکتری که کاربر تایپ کرده است (در صورت وجود) قرار گیرد و اگر کارکتری وجود نداشته باشد باید کرسر تغییری نکند.سپس اگر از آن کاراکتر چند تا داشتیم، باید با زدن همزمان ctrl F و کلید چپ یا راست ، بین این چند تا کاراکتر ، کرسر جابه جا شود. در نهایت اگر کاربر خواست کاراکتر دیگر را سرچ کند ، کافی است دوباره ctrl F را بزند.

برنامه سطح کاربر:

در این قسمت به اضافه کردن قابلیت `find_sum` می پردازیم. برای این کار در ابتدا بررسی می کنیم که اگر فقط نام دستور دستور وارد شده بود از برنامه خارج شویم. سپس در حلقه `while`، کارکترهای بعدی را بررسی می کنیم. `num` یک عدد مستقل وارد شده در رشته ورودی را نگه می دارد و `sum` نیز مجموع را نگه می دارد. در حلقه، هر گاه به یک کاراکتر عددی رسیدیم آن را در `num` ذخیره می کنیم. چنانچه کاراکتر بعدی آن، غیر عددی بود آن عدد را با `sum` جمع می کنیم و اگر کاراکتر بعدی عدد بود، `num` را در ۱۰ ضرب کرده و با عدد جدید جمع می کنیم و سپس همین منطق را برای کاراکتر بعدی پیاده می کنیم.

در انتها نیز چک می کنیم که اگر آخرین کاراکتر عدد بود، (یعنی `in_number` یک باقی مانده بود) عدد آخر را نیز با `sum` جمع می کنیم:


```

int main(int argc, char *argv[])
{
    if(argc < 2){
        printf(1, "\n");
        printf(1, "Usage: find_sum <string>\n");
        exit();
    }

    int sum = 0;
    int num = 0;
    int in_number = 0;
    char *current_char;
    int i;

    for(i = 1; i < argc; i++){
        current_char = argv[i];
        while(*current_char){
            if(*current_char >= '0' && *current_char <= '9'){
                num = num * 10 + (*current_char - '0');
                in_number = 1;
            } else {
                if(in_number){
                    sum += num;
                    num = 0;
                    in_number = 0;
                }
            }
            current_char++;
        }

        if(in_number){
            sum += num;
            num = 0;
            in_number = 0;
        }
    }
}

```

در ادامه باید ارقام متغیر sum را کاراکتر به کاراکتر جدا کرده و در یک آرایه ذخیره کنیم تا بتوانیم آن را در فایل بنویسیم. به همین منظور، در یک حلقه while ارقام آن با گرفتن باقیمانده بر ۱۰ جدا می‌کنیم و در آرایه buf_rev ذخیره می‌کنیم. سپس در حلقه while بعدی، آرایه buf_rev را معکوس کرده تا ترتیب درست ارقام sum بدست آید و این ترتیب درست را در آرایه buf ذخیره می‌کنیم.

همچنین در اول و آخر آرایه buf یک کاراکتر new_line قرار می‌دهیم که این دستور به شکل درستی در کنسول نمایش داده شود:

```

char buf[32];
int len = 0;

buf[len++] = '\n';

if(sum == 0){
    buf[len++] = '0';
} else {
    char buf_rev[32];
    int j = 0;

    while(sum > 0){
        buf_rev[j++] = (sum % 10) + '0';
        sum /= 10;
    }

    while(j > 0)
        buf[len++] = buf_rev[--j];
}

buf[len++] = '\n';

```

در ادامه باید آرایه buf را در فایل بنویسیم. به این منظور ابتدا با دستور unlike بررسی می‌کنیم که اگر فایل result از قبل وجود دارد پاک شود. سپس فایل را باز کرده و آرایه را در فایل می‌نویسیم:

```

unlink("result.txt");
int fd = open("result.txt", O_CREATE | O_WRONLY);
if(fd < 0){
    printf(1, "\n");
    printf(1, "cannot open result.txt\n");
    exit();
}

write(fd, buf, len);
close(fd);
printf(1, "\n");
exit();
}

```

همچنین باید عبارت `_find_sum\` را در متغیر `UPROGS` در `Makefile` اضافه کنیم:

```

UPROGS=\
    _find_sum\
    _cat\
    _echo\
    _forktest\
    _grep\
    _init\
    _kill\

```

قابلیت تکمیل خودکار:

در ابتدای کار نیاز به تعریف متغیرهای زیر داریم:

```
const char *cmds[] = {
    "README", "find_sum", "cat", "echo", "forktest", "grep", "init",
    "kill", "ln", "ls", "mkdir", "rm", "sh", "stressfs", "usertests",
    "wc", "zombie", "console", "cd"
};
#define CMDS_COUNT (sizeof(cmds)/sizeof(cmds[0]))
int tab_count = 0;
```

در آرایه cmds تمام دستورات ls به همراه دستور cd را اضافه می‌کنیم. در متغیر CMD_COUNTS تعداد آنها را نگه می‌داریم و در متغیر tab_counts تعداد دفعات فشردن دکمه تب را نگه می‌داریم.

در ادامه ۳ تابع کمکی تعریف می‌کنیم:

```
int has_prefix(const char *s, const char *p) {
    while (*p) {
        if (*s != *p) return 0;
        s++; p++;
    }
    return 1;
}

int collect_matches(const char *prefix, int *out_idx, int maxn) {
    int n = 0;
    for (int i = 0; i < (int)CMDS_COUNT; i++) {
        if (has_prefix(cmds[i], prefix)) {
            if (out_idx && n < maxn) out_idx[n] = i;
            n++;
        }
    }
    return n;
}

static void consputs(const char *s) {
    while (*s) consputc(*s++);
}
```

تابع has_prefix برابری دو string را تشخیص می‌دهد.

تابع collect_mathes تعداد انطباق‌های یک پیشوند در آرایه دستورات را برمی‌گرداند و همچنین ایندکس دستورات منطبق را نیز در out_idx ذخیره می‌کند.

تابع consputs نیز یک رشته را چاپ می‌کند.

در ادامه یک کیس جدید در switch case موجود در کنسول برای تب اضافه می‌کنیم.

سپس در قدم اول، باید ورودی کاربر را بدست بیاوریم. این ورودی در آرایه user_input ذخیره میشود:

```
switch (c)
{
case TAB_KEY: {

    if (input.e != input.end_pos) {
        break;
    }

    char user_input[INPUT_BUF];
    int len = 0;

    int pos = input.e - 1;
    if (pos < (int)input.w) {
        tab_count = 0;
        break;
    }

    while (pos >= (int)input.w) {
        char ch = input.buf[pos % INPUT_BUF];
        if (ch == ' ' || ch == '\n')
            break;
        pos--;
    }
    int start = pos + 1;

    for (int i = start; i < (int)input.e && len < INPUT_BUF-1; i++) {
        user_input[len++] = input.buf[i % INPUT_BUF];
    }
    user_input[len] = 0;
}
```

به این منظور، در ابتدا موقعیت ابتدای خط ورودی جدید را در بافر پیدا کرده و آن را در start ذخیره می‌کنیم و سپس از start تا input.e روی بافر حلقه می‌زنیم و آن را در user_input کپی می‌کنیم تا ورودی کاربر بدست آید.

سپس به کمک توابع کمکی تعریف شده، تعداد انطباق را در متغیر m و ایندکس‌های دستورات منطبق را در آرایه `cmd_indexes` ذخیره می‌کنیم:

```
tab_count++;  
  
int cmd_indexes[64];  
int m = collect_matches(user_input, cmd_indexes, 64);
```

حال اگر m برابر ۰ بود کاری نمی‌کنیم. اگر برابر ۱ بود و همچنین مقدار `tab_count` نیز یک بود، باید ادامه دستور پیدا شده را بنویسیم:

```
if (m == 0) {  
    tab_count = 0;  
}  
  
else if (m == 1) {  
    if (tab_count == 1) {  
        const char *full = cmds[cmd_indexes[0]];  
        const char *suffix = full + len;  
  
        while (*suffix) {  
            if (input.end_pos - input.r >= INPUT_BUF)  
                break;  
            input.buf[input.end_pos % INPUT_BUF] = *suffix;  
            input.end_pos++;  
            input.e++;  
            consputc(*suffix);  
            suffix++;  
        }  
        tab_count = 0;  
    }  
}
```

در این حالت، دستور پیدا شده در `full` ذخیره می‌شود و ادامه آن در `suffix` ذخیره می‌شود. سپس `suffix` در بافر وارد شده و روی صفحه نمایش داده می‌شود.

همچنین اگر m برابر ۲ بود باید به `tab_count` نگاه کنیم. اگر `tab_count` برابر ۱ بود کاری نمی‌کنیم و اگر بیشتر بود باید تمام دستورات پیدا شده را نمایش دهیم:

```
} else {  
    if (tab_count == 1) {  
    }  
    else {  
        consputc('\n');  
        for (int k = 0; k < m; k++) {  
            consputs(cmds[cmd_indexes[k]]);  
            consputc(' ');  
        }  
        tab_count = 0;  
    }  
}
```

س ۱۰) ULIB لیست آجکت‌های کتابخانه‌های مورد استفاده در فضای کاربر است. این کتابخانه‌ها به برنامه‌های کاربر لینک می‌شوند و وظیفه آنها فراهم کردن توابع کمکی برای هر برنامه user است. (ulib.o, usys.o, printf.o, umalloc.o)

UPROGS لیست برنامه‌های باینری فضای کاربر است که باید ساخته شوند است و در ایمج فایل سیستم (fs.img) قرار بگیرند تا بتوانند در xv6 اجرا شوند. (مثل _find_sum ، _cat و ...)

س ۱۱) در xv6 دستوراتی مانند ls و cat برنامه‌های فضای کاربر مستقل هستند و باینری‌شان در فایل سیستم وجود دارد اما دستور cd فایل اجرایی جدا ندارد و در فهرست برنامه‌ها نیست. تفاوت cd با این دستورها این است که cd داخل خود برنامه شل (والد) اجرا می‌شود و نه مثل بقیه دستورها در یک فرایند جدید. به بیان دیگر، هنگامی که دستوری مانند ls را می‌زنیم، سیستم‌عامل یک فرایند جدید (فرزند) با دستور fork() می‌سازد و این دستور در فرزند اجرا می‌شود. والد با wait() صبر می‌کند تا فرزند تمام شود و پس از پایان اجرای ls، فرزند از بین می‌رود و والد (شل) به کار خود ادامه می‌دهد. اما در cd این اتفاق نمی‌افتد و cd در خود شل (والد) اجرا می‌شود و شل مستقیماً chdir() را صدا می‌زند.

علت این اتفاق این است که اگر cd هم مثل ls یک برنامه جدا بود، یک فرزند برای آن ساخته می‌شد و در فرزند، سیستم کال chdir() می‌افتاد. پس مسیر جاری فرزند عوض می‌شد اما وقتی cd تمام می‌شد، فرزند از بین می‌رفت و والد در مسیر قبلی‌اش باقی می‌ماند. یعنی در ظاهر cd اجرا شده اما دایرکتوری شل تغییری نکرده. بنابراین cd یک برنامه جدا نیست.

س ۱۲) در سکتور نخست که همان boot sector است، محتوای فایل bootblock قرار دارد که از ترکیب bootasm.o و bootmain.o ساخته می‌شود. این کد، توسط BIOS در آدرس 0x7C00 بارگذاری و اجرا می‌شود و وظیفه آن بارگذاری کرنل از دیسک در حافظه است.


```
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
objdump -S bootblock.o > bootblock.asm
objcopy -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
```

س ۱۳) اغلب برنامه‌های xv6 در قالب فایل‌های باینری ELF نگهداری می‌شوند اما فایل بوت، یک فایل باینری تخت (flat) است که باید در سکتور ۵۱۲ بیتی اول جا شود و همانطور که در سوال قبل اشاره شد، توسط BIOS مستقیماً در آدرس 0x7C00 بارگذاری می‌شود. تفاوت فایل باینری تخت با فایل باینری عادی در این است که فایل باینری تخت، فقط شامل کد خام ماشین است و هیچ اطلاعات اضافی مانند هدر و سکشن‌ها و متادیتا ندارد. این فایل‌ها مستقیماً قابل بارگذاری در حافظه بوده و قابل اجرا توسط سخت‌افزار هستند.

علت استفاده از این نوع فایل این است که در مرحله بوت، هنوز هیچ سیستم‌عاملی وجود ندارد تا ساختار را تفسیر کند. به همین دلیل در بوت‌لودر باید از کد ماشین خالص استفاده شود تا BIOS بتواند آن را اجرا کند.

در انتها با دستور `objdump -S bootblock.o > bootblock.asm` فایل بوت را (bootblock.o) به فایل اسمبلی تبدیل می‌کنیم. (در گیت‌هاب موجود است).

س ۱۴) در فرایند ساخت xv6، از `objcopy` استفاده می‌شود تا خروجی لینک بوت که قالب ELF دارد به یک باینری تخت (flat) تبدیل شود. با این دستور، هدر و سکشن‌های اضافه ELF حذف می‌شوند و فقط کد `.text` باقی می‌ماند تا بوت، دقیقاً به صورت سکتور ۵۱۲ بیتی روی دیسک نوشته شود و BIOS بتواند آن را مستقیماً در 0x7C00 بارگذاری و اجرا کند.

خط زیر که در Makefile وجود دارد این کار را انجام می‌دهد:

```
objcopy -S -O binary -j .text bootblock.o bootblock
```

س15) در معماری xv6 ابتدا فرآیند بوت انجام میشود که ابتدا از زبان اسمبلی استفاده میشود. با این کار رجیسترها و استکها مقدار دهی میشوند و پردازنده به مود **protected** منتقل میشود که 32 بیتی است. اکنون میتوان از کد C برای ادامه بوت کردن استفاده کرد که 32 بیتی است و وظیفه آن خواندن از دیسک و بارگذاری در حافظه است زیرا زبان C سطح بالاتری دارد و برای وظایف پیچیده تر مناسب تر است.

س16)

رجیستر **general purpose**: این رجیسترها عموماً برای ذخیره محاسبات در پردازنده رجیستر **Accumulator** که همان رجیستری است که محاسبات **ALU** را ذخیره میکند.

رجیستر **segment**: میتوان به رجیستر **Stack Segment** اشاره کرد که وظیفه نگهداری اطلاعات مربوط به استک را مثل آدرس بازگشت را برعهده دارد.

رجیستر **Status**: این رجیسترها وظیفه نگهداری بیت های **flag** را دارند مثلاً رجیستر **EFLAG** که 32 بیتی است و هر بیت آن نشان دهنده یک وضعیت می باشد مانند وضعیت **overflow** یا **carry** یا **privilege level** و مدیریت وقفه و...

رجیستر کنترلی: این رجیسترها نقش مهمی در رفتار کلی پردازنده و ارتباط آن با سایر اجزای سیستم است. برای مثال رجیستر **CR2** آدرس خطای **page fault** را ذخیره میکند.

س17) در این پردازنده سیستم عامل ابتدا برای بوت شدن به حالت **real mode** میرود زیرا فرآیند بوت راحت تر میشود و نیازی به جدول صفحه و سیستم حفاظت نیست. هم چنین پردازنده های قبلی اینتل مانند 8086 هم همین طور بوده و از این نظر میتوان گفت سازگار تر است. و چون DOS برای پردازنده 8086 بوده ،آن هم در همین مود بوت میشد.

نقص اصلی این مود این است که سطح حفاظت بسیار پایین می آید و یک برنامه ممکن است با دسترسی غیر مجاز به بخشی از حافظه مشکل ساز شود. و مورد بعدی محدودیت حافظه در این مود است که فقط 20 بیت آدرس فیزیکی داریم.

در سیستم ARM به این شکل نیست و پردازنده از همان ابتدا در حالت **supervisor mode** اجرا میشود و ایمن و مدرن است.

در RISC V هم ما 3 سطح داریم و بوت در سطح **Machine mode** اجرا میشود و مانند x86 نیست و از همان ابتدا **protection** دارد و از قابلیت های دیگر پشتیبانی میشود.

س18) این مود برای x86 یک مودی است که در آن حفاظت و مدیریت پیشرفته حافظه و منابع اجرا میشود. و از ویژگی های کلیدی این مود میتوان به محافظت از حافظه ، سطوح دسترسی ، مدیریت بهتر وقفه و استثناها و پشتیبانی از حافظه مجازی و **paging** اشاره کرد.

سیستم عامل پس از بوت شدن ، برای فعال سازی امکانات مدرن تر سیستم عامل نیاز دارد ابتدا به این مود بیايد. برای این کار روی رجیستر های مختلف بایستی عملیات هایی انجام دهد.پس از آن

سیستم عامل در این مود میماند تا زمانی که وارد مود کاربر شود و یک لایه پایین تر بیاید و سطح دسترسی تنظیم شود.

س19) علت این انتخاب این است که در آدرس های 1 مگابایت اولیه ، معمولا مربوط به BIOS سیستم عامل و بخش هایی برای بوت لودر آن. و اولین جایی که در حافظه مناسب است همین شروع از آدرس 1 مگابایت است تا هسته روی سیستم اجرا شود. آدرس های بالاتر ممکن است با بخش IO تداخل پیدا کند.

اگر از آدرس 0 شروع می کردیم، هسته سیستم عامل با بخش BIOS و بخش های پایه ای تر سیستم عامل تداخل پیدا می کرد.

س20) برای مشاهده breakpoint ها می توانیم از دستور `info breakpoints` استفاده کنیم. این دستور اطلاعات کاملی از breakpoint ها به ما ارائه می دهد.

س21) برای حذف یک breakpoint خاص، می توانیم از دستور `delete <num>` استفاده کنیم.

مثلا اگر بخواهیم breakpoint شماره ۱ را حذف کنیم از `delete 1` استفاده می کنیم.

یا مثلا اگر بخواهیم شماره های ۱ و ۲ و ۳ را حذف کنیم از `delete 1 2 3` استفاده می کنیم.

همچنین اگر بخواهیم همه breakpoint ها را حذف کنیم از دستور `delete` استفاده می کنیم.

س22) `bt` مخفف “backtrace” است که `call stack` را نشان میدهد. یعنی لیستی از توابع که فراخوانی شده اند (و هنوز `return` نشده اند). خروجی آن برای مثال به صورت زیر خواهد بود:

```
(gdb) bt
#0  main () at main.c:19
```

عدد 0 : frame number هست که تابع فعلی که پردازنده روی آن متوقف شده است

main () : نام آن تابع

19:at main.c: فایل که تابع در آن است و خطی که پردازنده متوقف شده است.

س ۲۳) دستور X برای خواندن و بررسی محتویات حافظه است. استفاده از دستور X به صورت زیر است:

x/FMT ADDRESS

که FMT به ترتیب تعداد واحد ها فرمت و اندازه هر واحد است.

دستور print مقدار عبارت EXP (یا هر متغیری که بعنوان EXP به آن داده شود) را چاپ میکند و بصورت زیر کاربرد دارد:

print [[OPTION]... --] [/FMT] [EXP]

حالا برای نمایش یک ثبات خاص باید از دستور print استفاده کرد به این صورت که بعد از کلمه print یک \$ گذاشته و سپس نام ثبات را می آوریم. مثلا برای ثبات eax:

Print \$eax

س ۲۴) برای نمایش وضعیت ثبات ها در GDB میتوان از دستور `info registers` استفاده کرد. این دستور همه ثبات های عمومی 'شمارنده برنامه و فلگ ها و مقادیرشان را نمایش میدهد.

```
(gdb) info registers
eax            0x80104610      -2146417136
ecx            0x0          0
edx            0x1f0        496
ebx            0x10074      65652
esp            0x80116d70    0x80116d70
ebp            0x7bf8       0x7bf8
esi            0x10074      65652
edi            0x116d70     1142128
eip            0x80104610    0x80104610 <main>
eflags         0x86         [ IOPL=0 SF PF ]
cs             0x8          8
ss             0x10         16
ds             0x10         16
es             0x10         16
fs             0x0          0
gs             0x0          0
fs_base        0x0          0
gs_base        0x0          0
k_gs_base      0x0          0
cr0            0x80010011    [ PG WP ET PE ]
cr2            0x0          0
cr3            0x10a000     [ PDBR=266 PCID=0 ]
cr4            0x10         [ PSE ]
cr8            0x0          0
```

برای نمایش متغیر های محلی از دستور `info locals` استفاده میشود که متغیر های محلی مربوط به فریم فعلی را نمایش میدهد.

مثال:

```
(gdb) break main
Breakpoint 1 at 0x80104610: file main.c, line 19.
(gdb) break consoleintr
Breakpoint 2 at 0x80101448: file console.c, line 576.
(gdb) continue
Continuing.
=> 0x80101448 <consoleintr>:    push    %ebp

Thread 1 hit Breakpoint 2, consoleintr (getc=0x80106d48 <uartgetc>) at console.c:576
576      {
(gdb) info locals
c = <optimized out>
doprocdump = <optimized out>
```

همانطور که مشاهده میشود این دستور متغیر های محلی تابع `consoleintr` را نمایش داده است.

ESI و EDI و رجیستر عمومی (general-purpose) در x86 (32-bit) هستند. نام‌های تاریخی‌شان «source index» و «destination index» است چون برای دستورات رشته‌ای (مثل movs, lods, stos, scas) به عنوان اندیس منبع و مقصد استفاده می‌شدند. همانطور که گفته شد ESI شاخص برای منبع هست که آدرس مبدا منبع داده‌هایی که لازم هست را نگه می‌دارد. همچنین EDI شاخص مقصد است که آدرس مقصد داده‌ها را نگهداری میکند.

س ۲۵) این ساختار یک بافر حلقوی دارد که سایز آن در فایل کنسول تعریف شده است (INPUT_BUF). برای استفاده از این بافر حلقوی هر دفعه باید مد گرفت به سایز آن. R اندیس خواندن است از بافر که سیستم از آن استفاده میکند تا ورودی کاربر را از ابتدای آن بخواند و با خواندن هر کاراکتر یکی به مقدار آن اضافه میشود. W موقعیت نوشتن در بافر است که برای ورودی جدید به کار میرود و پس از هر دستور آپدیت میشود. E موقعیت ویرایش کاراکتر در بافر است که برای هندل کردن شرایطی مثل وارد کردن کاراکتر جدید استفاده از backspace و arrow key ها به کار می‌آید و تغییر میکند.

مقدار R زمانی تغییر میکند که هسته می‌خواهد ورودی داده شده کاربر را بخواند و پس از آن مقدار آن را آپدیت میکند و به جای آخرین کاراکتر موجود در بافر برای آن دستور میبرد. مقدار W با زدن enter به جلو میرود و در آخر دستور قبل قرار میگیرد تا دستور بعدی از آنجا شروع به نوشتن کند. مقدار e با زدن enter مساوی w خواهد شد و با هر اضافه کردن کاراکتر توسط کاربر افزوده و با پاک کردن آن کاهش می‌یابد (همچنین با استفاده از آپشن‌های دیگر مثل کلید‌های اشاره گر مقدار آن میتواند تغییر کند).

س ۲۶) دستور src layout در رابط گرافیکی GDB سورس کد به زبان C را نشان میدهد که قابل پیمایش است. مثال:

```
console.c
567 }
568 return n;
569 }
570
571 static void consputs(const char *s) {
572     while (*s) consputc(*s++);
573 }
574
575 void consoleintr(int (*getc)(void))
B+> 576 {
577     int c, doprocdump = 0;
578     acquire(&cons.lock);
579     if (input.e > input.end_pos)
580     {
581         input.end_pos = input.e;
582     }
583
584     while ((c = getc()) >= 0)
585     {
586
remote Thread 1.1 (src) In: consoleintr L576 PC: 0x80101448
```

دستور asm layout در رابط گرافیکی GDB سورس کد که به زبان اسمبلی تبدیل شده است (توسط کامپایلر) را نشان میدهد که قابل پیمایش است. مثال:

```
0x80101449 <consoleintr+1> mov    %esp,%ebp
0x8010144b <consoleintr+3> push  %edi
0x8010144c <consoleintr+4> push  %esi
0x8010144d <consoleintr+5> push  %ebx
0x8010144e <consoleintr+6> sub    $0x1b8,%esp
0x80101454 <consoleintr+12> mov    0x8(%ebp),%eax
0x80101457 <consoleintr+15> mov    %eax,-0x19c(%ebp)
0x8010145d <consoleintr+21> push  $0x801107c0
0x80101462 <consoleintr+26> call   0x801059e0 <acquire>
0x80101467 <consoleintr+31> mov    0x80109088,%eax
0x8010146c <consoleintr+36> add    $0x10,%esp
0x8010146f <consoleintr+39> cmp    %eax,0x8010908c
0x80101475 <consoleintr+45> jae    0x8010147c <consoleintr+52>
0x80101477 <consoleintr+47> mov    %eax,0x8010908c
0x8010147c <consoleintr+52> movl   $0x0,-0x1a0(%ebp)
0x80101486 <consoleintr+62> mov    -0x19c(%ebp),%eax
0x8010148c <consoleintr+68> call   *%eax
0x8010148e <consoleintr+70> mov    %eax,%ebx
0x80101490 <consoleintr+72> test   %eax,%eax
0x80101492 <consoleintr+74> js     0x80101604 <consoleintr+444>

remote Thread 1.1 (asm) In: consoleintr L576 PC: 0x80101448
```

س ۲۷) برای حرکت به سمت بالا یعنی به توابعی که تابع فعلی را فراخوانی کرده اند میتوان از دستور `up <number>` استفاده کرد که به میزان عدد وارد شده به همان تعداد بار به سمت بالا حرکت میکند.

برای حرکت به سمت پایین یعنی توابعی که تابع فعلی آنها را فراخوانی میکند میتوان از دستور `down` `<number>` استفاده کرد که به میزان عدد وارد شده به همان تعداد بار به سمت پایین حرکت خواهد کرد.

برای جابجایی به یک فریم خاص از زنجیره فراخوانی ها میتوان از دستور `frame <number>` استفاده کرد با توجه به این نکته که شماره فریم تابع فعلی صفر است و با توجه به عدد وارد شده به همان فریم از زنجیره فراخوانی میرود.